哈爾濱工程大學

第 06 讲 / 全 17 讲

C 语言 while 循环用法详解

当循环次数由人决定，而不是由你决定

while 三要素

初值在前、条件在括号里、步进在循环体内

do-while 先做后判

至少执行一次，菜单程序的最佳搭档

break 与 continue

一个直接收工，一个跳过这趟继续下一趟

经典算法

gcd 、数字反转、判回文、二分逼近

哈尔滨工程大学 · C 语言程序设计　主讲教师：郎大鹏

本节目标

哈爾濱工程大學

1. 说清 for 与 while 的分工：次数已知用 for ，次数未知用 while
2. 写出 while 三要素：循环前给初值、括号里写条件、循环体内做步进
3. 把任意一个 for 循环，等价改写成 while 循环
4. 用 break 跳出循环、用 continue 跳过本次，并躲开 continue 吃掉步进的坑
5. 用哨兵值和 scanf 的返回值，优雅地让循环停下来
6. 用 while 写出 gcd 、数字反转、判回文、二分逼近等经典算法

01

while 基础

把循环的开关，从计数器手里拿回来

三要素在哪里 · 与 for 的对照 · 别写成死循环 · do-while

哈爾濱工程大学

为什么学了 for ，还要学 while

哈尔滨工程大学

for 擅长对付「早就知道要转几圈」的事情；

可现实里还有一大类问题——动手敲代码时根本不知道要转几圈。

已知次数： for 就够用

```
int i;
for (i = 1; i <= 5; i++)
{
    printf("%d ", i);
}
// 输出: 1 2 3 4 5
```

for ：圈数钉死在括号里

未知次数：交给 while

```
int target = 37, guess = 0;
while (guess != target)
{
    printf(" 请猜一个数: ");
    scanf("%d", &guess);
}
```

while ：转几圈？用户说了算

while 三要素：一个都不能少

哈爾濱工程大學

```c
#include <stdio.h>
int main(void)
{
    int i = 1;          // ① 初值：while 之前
    while (i <= 5)      // ② 条件：每轮开头检查
    {
        printf("%d ", i);
        i = i + 1;      // ③ 步进：让条件走向假
    }
    return 0;
}
// 屏幕输出：1 2 3 4 5
```

① 初值：循环之前

循环变量必须先有自己的值，条件才有东西可判断。写 int i = 1; 的位置在 while 上面。

② 条件：括号里面

每一轮开始先算这个表达式。结果为真才执行循环体，为假立刻结束。

③ 步进：循环体内

写在花括号里，每轮让变量朝「条件变假」的方向走一步——它是循环的刹车。

while 与 for ：就是搬个家

哈爾濱工程大學 1953 HARBIN ENGINEERING UNIVERSITY

写法一： for —— 三件事挤在括号里

```
int i;
for (i = 1; i <= 5; i++)
{
    printf("%d ", i);
}
// 输出: 1 2 3 4 5
```

写法二： while —— 三件事各就各位

```
int i = 1;
while (i <= 5)
{
    printf("%d ", i);
    i = i + 1;
}
```

结论：逻辑一模一样，只是「初值、条件、步进」三件东西搬家了。

① 初值

for 版: `for (i = 1;`

while 版: `int i = 1;`

② 条件

for 版: `i <= 5;`

while 版: `while (i <= 5)`

③ 步进

for 版: `i++)`

while 版: `i = i + 1;`

计数循环：算 1 到 100 的和

```c
#include <stdio.h>
int main(void)
{
    int i = 1, sum = 0;      // sum 是累加器
    while (i <= 100)
    {
        sum = sum + i;       // 边算边存
        i = i + 1;           // 步进
    }
    printf("和 = %d", sum);
    return 0;
}
```

累加器必须先清零

sum = 0 一定要写。不写的话 sum 里是上次留下的垃圾值，加出来全是错的。

逐轮实测

```
i=1     sum = 0+1 = 1
i=2     sum = 1+2 = 3
i=3     sum = 3+3 = 6
...     ...
i=100   sum = 4950+100
i=101   条件为假，退出
```

死循环的预防：谁让条件变假

哈尔滨工程大学

✗ 忘写步进：转不停

```
int i = 1;
while (i <= 5)
{
    printf("%d ", i);
}
// i 永远是 1，停不下来
```

✓ 补上步进：五圈就停

```
int i = 1;
while (i <= 5)
{
    printf("%d ", i);
    i = i + 1;      // 关键这一句
}
```

死循环长什么样

条件永远为真，程序就一直转下去，屏幕刷得停不下来，看着像卡死了。

所以每写一个 while ，先指着代码问自己一句：到底哪一句让条件变假？

do-while ：先做一次，再判断

```c
#include <stdio.h>
int main(void)
{
    int choice = 0;
    do
    {
        printf("1. 查询  2. 存款  0. 退出: ");
        scanf("%d", &choice);
        if (choice == 1) printf(" 查询余额 ");
        else if (choice == 2) printf(" 存好了 ");
    } while (choice != 0);   //  分号别丢
}
```

形状：先做，后判

先把循环体执行一遍，再回头看条件。真就回去再来一次，假就往下走。

为什么菜单非它不可

菜单至少得显示一次，用户才知道有哪些选项。用 while 有可能一次都不显示。

结尾那个分号

do-while 的 while(条件) 后面必须带分号，这是它跟 while 最显眼的长相差别。

02

循环控制

让循环知道，什么时候该停

break 跳出 · continue 跳过 · 哨兵值 · scanf 返回值 · 四问

哈爾濱工程大学

break ：立刻跳出整个循环

```
int i = 1, sum = 0;
while (i <= 10)
{
    if (i == 5)
        break;      // 到 5 就收工
    sum = sum + i;
    i = i + 1;
}
printf("%d", sum);   // 输出 10
```

1+2+3+4 = 10 —— 第 5 圈没走完
循环被腰斩，后面的 6 到 10 再也不会跑。

它做了什么

立刻结束整个循环，程序跳到循环后面的第一句，条件也不再检查。

必须配 if 用

不带 if 的 break 一定在第一圈就执行，等于把循环写废了。

只跳最里面一层

以后写嵌套循环时， break 只负责它所在的那一层，外层照旧转。

continue ：跳过这一趟


```
int i = 0, sum = 0;
while (i < 10)
{
    i = i + 1;          //  步进放最前面
    if (i == 5)
        continue;       // 5  这趟不要
    sum = sum + i;
}
printf("%d", sum);      //  输出  50
```

把 i = i + 1; 挪到 continue 后面试试——

i 卡在 5 上永远加不上去，死循环。

它做了什么

结束这一轮，直接回去重新判断条件，下一轮照常开始。

什么时候用

这一份数据不参与处理，剩下的还要接着做：跳过 5 、跳过负数、跳过空行。

血泪坑

步进必须写在 continue 前面。写在后面，这一轮的步进就被永久跳过了。

break 与 continue ：断 vs 绕

哈爾濱工程大學

break ：只输出 1 2 3 4

```
int i = 1;
while (i <= 10)
{
    if (i == 5) break;
    printf("%d ", i);
    i = i + 1;
}
```

continue ：没有 5 ，其余全有

```
int i = 0;
while (i < 10)
{
    i = i + 1;
    if (i == 5) continue;
    printf("%d ", i);
}
```

break = 断

一刀切断，整条循环连同后面的轮次全部作废。

适合「找到了，不用再找了」。

continue = 绕

绕开这一趟，循环继续转，下一轮该干的事照干。

适合「这个不合格，跳过它」。

哨兵值：用一个特殊值喊停

```c
#include <stdio.h>
int main(void)
{
    int x, sum = 0;
    scanf("%d", &x);     //  先读一个
    while (x != -1)      //  哨兵收尾
    {
        sum = sum + x;
        scanf("%d", &x); //  再读下一个
    }
    printf(" 和  = %d", sum);
}
```

哨兵值是什么

一个正常数据里绝不会出现的值，专门用来喊停。输入非负数时，常用 -1 。

为什么读两次

循环前读一次，条件才有东西可判断；循环里再读一次，才能推进到下一个数。

顺序不能反

先判断、后累加， -1 才不会混进 sum 里。写反了结果会莫名多减 1 。

让 scanf 的返回值当条件


```c
#include <stdio.h>
int main(void)
{
    int x, sum = 0;
    while (scanf("%d", &x) == 1)
    {
        sum = sum + x;
    }
    printf("和 = %d", sum);
}
```

scanf 课只提了一句的「返回值」，

今天在这里正式上班了。

返回值 = 读到了几个

成功读入一个整数，scanf 就返回 1；所以 == 1 就是「这次读成功了」。

读失败返回什么

碰到非数字返回 0；输入彻底结束返回 EOF，也就是 -1。两者都不等于 1。

为什么这样最省心

数据读完、格式不对，循环都自己停，不用再约定一个哨兵值。

循环设计四问

动手之前先在草稿纸上回答这四问，答不全就别急着敲键盘。

① 初始：从哪儿开始

进循环之前，循环变量是什么值？这个值要能通过第一次判断。

② 条件：转到何时

括号里写哪个表达式？它为真就继续转，为假就收工。

③ 步进：每轮走多远

循环体里哪一句让变量往终点挪一步？挪的方向对不对？

④ 终止：谁让它停

条件什么时候变假？答不出来，这就是一个死循环。

拿猜数字套一遍

初始 `guess = 0` ；条件 `guess != 37` ；步进 每轮读一次 `guess` ；终止 猜中就退出。

四问答完，骨架就自己长出来了：初值； `while ( 条件 ) { 循环体；步进 }`

03

经典算法

while 是算法的老搭档 —— 不知道几轮，正好用它

辗转相除 · 数字反转 · 位数 · 二分逼近 · 判回文 · 菜单

哈爾濱工程大学

辗转相除法求最大公约数

```
int a = 48, b = 18, t;
while (b != 0)          // b 归零就收工
{
    t = a % b;          //  求余数
    a = b;              //  除数上位
    b = t;              //  余数当除数
}
printf("gcd = %d", a);    //  输出 6
```

48 与 18 的推演

```
48 % 18 = 12    a=18, b=12
18 % 12 = 6     a=12, b=6
12 % 6  = 0     a=6,  b=0   停
```

思路

两数相除取余，除数升为被除数，余数降为除数，一轮轮滚下去。

为什么一定会停

余数一定比除数小，每一轮都在变小，最后必然落到 0。

答案留在哪

循环结束时 b 已经被清零，真正的答案留在 a 里。

数字反转：123 变 321

```
int n = 123, d, rev = 0;
while (n != 0)
{
    d = n % 10;          //  取末位
    rev = rev * 10 + d; //  接到尾巴上
    n = n / 10;          //  砍掉末位
}
printf("%d", rev);       //  输出  321
```

两个动作

n % 10 取末位，n / 10 砍末位。整数除法天然丢小数，正好当砍刀。

rev 为什么要乘 10

乘 10 是把已有的位往左挪一格，空出来的个位再放进新的数。

123 是怎么翻过来的

第 1 轮	d=3	rev=3	n=12	
第 2 轮	d=2	rev=32	n=1	
第 3 轮	d=1	rev=321	n=0	停

循环什么时候停

n 被砍到 0 就没数可取了。0 本身转不了，这是它的特例。

数一数它是几位数

```
int n = 2026, count = 0;
while (n != 0)
{
    n = n / 10;          //  砍掉一位
    count = count + 1;   //  记一笔
}
printf(" 位数  = %d", count);    // 4
```

0 是个例外

如果输入就是 0 ，循环一次都不转， count 停在 0 。

所以要么用 do-while ，要么事先判断 n == 0 时位数就是 1 。

核心就一句

砍一刀记一笔，砍不动了就是砍完了。跟反转只差「把末位接起来」这一步。

走一遍 2026

2026 → 202 → 20 → 2 → 0

砍了四次，所以 4 位。

负数也没问题

n = -2026 时整数除法照样朝 0 收敛，最后一样是 0 ，位数还是 4 。

二分逼近：算根号 2

```
int n = 2;
double lo = 0, hi = n, mid;
mid = (lo + hi) / 2;
while (hi - lo > 0.000001)   //  区间还大
{
    mid = (lo + hi) / 2;
    if (mid * mid > n)
        hi = mid;       //  猜大了
    else
        lo = mid;       //  猜小了
}
printf("%f", mid);    // 1.414214
```

把答案关起来

答案一定落在 lo 到 hi 之间。每次从正中间切一刀，区间砍掉一半。

浮点别用 ==

浮点数几乎不可能恰好相等，所以条件写成「区间够不够小」，不是「等不等」。

正好用 while

要走几轮事先算不出来，只认「精度够了没有」——for 反而别扭。

判回文数：翻过来看看


```
int n = 12321, m, rev = 0;
m = n;                  //  留一份原数
while (m != 0)
{
    rev = rev * 10 + m % 10;
    m = m / 10;
}
if (rev == n)
    printf(" 是回文数 ");
else
    printf(" 不是回文数 ");
```

正着读反着读都一样的数，就叫回文数。

复用上一页的本事

反转的代码原封不动，只在最后多一步：把反过来的数和原数比一比。

为什么要 m = n

循环会把 m 砍成 0 ，原数早就没了。先留一份，最后才有得比。

小提醒

负数一般不谈回文；末尾是 0 的数（如 120 ）翻过来是 21 ，必然不等。

菜单程序：do-while 的主场

```
int choice = -1;
do
{
    printf("1. 查询  2. 存款  0. 退出: ");
    scanf("%d", &choice);
    if (choice == 1)
        printf(" 查询余额 ");
    else if (choice == 2)
        printf(" 存款成功 ");
    else if (choice != 0)
        printf(" 没有这个选项 ");
} while (choice != 0);
```

为什么非它不可

菜单至少要显示一次，用户才知道有哪些选项。先做后判，正好。

if 链也能分岔

一层层 else if 比下去，分支照样分得清楚，不需要别的语句。

0 就是出口

用户敲 0 ，条件 choice != 0 变假，循环结束，程序收工。

猜数字游戏


```c
#include <stdio.h>
int main(void)
{
    int target = 37, guess = 0;
    while (guess != target)   //  没猜中就继续
    {
        printf(" 请猜一个数： ");
        scanf("%d", &guess);
    }
    printf(" 猜中了！ ");
}
```

运行结果

```
请猜一个数： 50
请猜一个数： 25
请猜一个数： 37
猜中了！
```

第一次「程序等你」

读不懂 for 写这个题：你根本不知道该转几圈。猜 3 次还是 30 次，全看用户。

while 把「停不停」交给 guess != target ，代码写得跟人的想法一模一样。

累加直到输入 -1

例 2 · 哨兵值收尾

哈爾濱工程大學

```c
#include <stdio.h>
int main(void)
{
    int x, sum = 0;
    scanf("%d", &x);      // 先读第一个
    while (x != -1)       // -1 就停
    {
        sum = sum + x;
        scanf("%d", &x); // 再读下一个
    }
    printf("和 = %d", sum);
}
```

运行结果

输入： 10 20 30 -1

和 = 60

-1 只是暗号，不进 sum

两处 scanf 的分工

循环外那句负责开张，让条件第一次有数可判；循环里那句负责推进，每轮换一个新数。

循环里那句一旦漏了，x 不变，条件永远为真。

辗转相除求 gcd

例 3 · 经典 while 骨架

哈爾濱工程大學

```c
#include <stdio.h>
int main(void)
{
    int a, b, t;
    scanf("%d %d", &a, &b);
    while (b != 0)      // 除数为 0 就停
    {
        t = a % b;
        a = b;
        b = t;
    }
    printf("gcd = %d", a);
}
```

运行结果

输入： 48 18

gcd = 6

48 和 18 都能被 6 整除

三个变量跳一支舞

t 先接住余数，再把 b 挪进 a、把 t 挪进 b。顺序错一步，结果全乱。

余数越滚越小，最后必然出现 0——循环自己就知道该收工了。

数字反转

例 4 · 取末位与砍末位

哈尔滨工程大学

```c
#include <stdio.h>
int main(void)
{
    int n, d, rev = 0;
    scanf("%d", &n);
    while (n != 0)
    {
        d = n % 10;          // 取末位
        rev = rev * 10 + d; // 接上去
        n = n / 10;          // 砍末位
    }
    printf("%d", rev);
}
```

运行结果

输入： 123

输出： 321

输入 100 呢？输出 1

为什么 100 会变成 1

rev 是整数，前面的 0 存不住：3 位数字里末尾的零会被自然丢掉。

这不是 bug ，是整数类型的本性，做判回文时要记住这一点。

简易菜单

例 5 • do-while 收尾

哈爾濱工程大學

```c
#include <stdio.h>
int main(void)
{
    int choice = -1;
    do
    {
        printf("1. 查询  2. 存款  0. 退出: ");
        scanf("%d", &choice);
        if (choice == 1) printf(" 查到了 ");
        else if (choice == 2) printf(" 存好了 ");
    } while (choice != 0);
}
```

运行结果

```
1. 查询  2. 存款  0. 退出: 1
1. 查询  2. 存款  0. 退出: 0
```

敲 0 才跳出循环

菜单为什么必须 do-while

换成 while ，就要在循环外先把菜单抄一遍，代码白白多出一份。

do-while 天然就是「先亮菜单，再看选了什么」。

六个最容易踩的坑

哈爾濱工程大學

- ✗ **忘写步进** 循环里没有让变量变化的那一句，条件永远为真
- ✗ **条件永远为真** while (1) 却没配 break ，只能用任务管理器强杀
- ✗ **continue 跳过步进** 步进写在 continue 后面，某些轮次永远走不到它
- ✗ **while 后误加分号** while (x < 5); 变成空循环，下面的花括号跟它没关系
- ✗ **do-while 漏了结尾分号** } while (x != 0) 后面少个分号，编译直接报错
- ✗ **循环内 scanf 放错位置** 放到循环外或条件里，数据读不进来，循环空转

小结

次数已知用 for ，次数未知用 while —— 这是选句型的第一个判断。

三要素缺一不可：初值在循环前、条件在括号里、步进在循环体内。

do-while 先做后判； break 立刻收工； continue 只绕开这一趟。

下一讲预告

函数：把一段代码装进盒子

写完一次，以后随叫随到 —— 今天这些 while ，将来都会住进盒子里。

哈爾濱工程大学